Teradata Ops Copilot — Implementation Runbook

Version: 1.0
Covers: All five scaffold phases (Models → DDL/Repos → Workflow → MCP/Services → Prompts/Tests)
Total scaffold size: ~7,100 lines across 71 source files
Test suite: 101 tests, all passing against stubbed repositories (no live Teradata required)

Table of Contents

---

1. Prerequisites and Environment Setup

1.1 Runtime requirements

Requirement	Version	Notes
Python	3.11 or 3.12	3.12 used in scaffold development and test runs
Teradata Vantage	17.x or later	For DDL execution and live repository calls
Teradata Python driver	`teradatasql >= 17.20`	Install from PyPI; requires Teradata ODBC runtime on host
Teradata Enterprise MCP server	Platform-specific	URL configured via `MCP_ENDPOINT_URL` env var
Anthropic API access	Any tier	Optional for Phase 4–5 LLM integration; scaffold functions without it

1.2 Python environment

```
# Create and activate virtual environment
python3.12 -m venv .venv
source .venv/bin/activate

# Install production dependencies
pip install -r requirements.txt

# Install development/test dependencies
pip install -r requirements-dev.txt

# Verify core imports work (no Teradata driver required for this check)
python3 -c "from app.models.domain import UserRequest, ContextPack; print('OK')"
```

1.3 Environment variables

Create a `.env` file at the project root. All variables have defaults that allow the scaffold to run in stub mode (no live Teradata or MCP connection required):

```
# Teradata connection
TD_HOST=your-teradata-host
TD_USERNAME=ops_copilot_svc
TD_PASSWORD=changeme
TD_DATABASE=context
TD_LOGMECH=TD2
TD_ENCRYPTDATA=true
TD_CONNECT_TIMEOUT=30
```

```
# MCP (leave blank for stub mode)
MCP_ENDPOINT_URL=
MCP_TIMEOUT_SECONDS=30.0

# Entity resolution thresholds
ENTITY_MIN_AUTO_RESOLVE_CONFIDENCE=0.85
ENTITY_AMBIGUITY_MARGIN=0.05

# Root cause scoring
MIN_ACTION_CONFIDENCE=0.60
DEPENDENCY_CHAIN_MAX_DEPTH=3

# LLM (leave blank to disable LLM summarization)
ANTHROPIC_API_KEY=
LLM_MODEL=claude-sonnet-4-6
LLM_MAX_TOKENS=1024

# App
APP_HOST=0.0.0.0
APP_PORT=8000
DEBUG=false
LOG_LEVEL=INFO
```

> **Security note:** Never commit `.env` to source control. Add it to `.gitignore`. For production Wells Fargo deployment, source secrets from your approved secrets-management platform (CyberArk, Vault, or equivalent) rather than a `.env` file.

---

2. Repository Initialisation

2.1 Unpack the phase zips in order

Each phase zip is **cumulative** (Phase 1–2 includes Phase 1 files, etc.) but Phase 4–5 is delivered as a **delta** (Phase 4–5 only contains the new files from those phases). Lay them out as follows:

```
teradata-ops-copilot/          ← your repo root
├── app/                       ← from phase1-2-3 zip
│   ├── models/                ← Phase 1
│   ├── sql/                   ← Phase 2
│   ├── repositories/          ← Phase 2
│   ├── workflows/             ← Phase 3
│   │   └── nodes/
│   ├── mcp/                   ← Phase 4 (delta zip)
```

```
│   │   └── adapters/
│   ├── services/              ← Phase 4 (delta zip)
│   ├── core/                  ← Phase 4 (delta zip)
│   ├── api/                   ← Phase 4 (delta zip)
│   ├── prompts/               ← Phase 5 (delta zip)
│   ├── tests/                 ← Phase 5 (delta zip)
│   └── main.py                ← Phase 4 (delta zip)
├── requirements.txt           ← Phase 4 (delta zip)
├── requirements-dev.txt       ← Phase 4 (delta zip)
└── README.md
```

2.2 Verify the full file inventory

```
find app/ -name "*.py" -o -name "*.sql" -o -name "*.txt" | sort | wc -l
# Expected: ~71 files
```

2.3 Run the test suite immediately (no Teradata required)

```
python3 -m pytest app/tests/ -v
# Expected: 101 passed, 0 failed
```

If any test fails before you have touched any code, the most likely cause is a missing dependency. Re-run `pip install -r requirements-dev.txt` and retry.

---

3. Phase 1 — Project Structure and Pydantic Models

Files: `app/models/domain.py`, `workflow.py`, `audit.py`, `policy.py`, `mcp.py`
Status: Complete and production-ready as-is. No TODOs.

3.1 What was built

Model	File	Key validators
`UserRequest`	`domain.py`	`raw_text` non-empty, whitespace-stripped
`ResolvedEntity`, `EntityCandidate`	`domain.py`	`confidence` bounded `[0.0, 1.0]`
`PolicyDecision`	`domain.py`	Enum-typed `status`

`ContextPack`	`domain.py`	All list fields default to `[]`
`ToolInvocation`	`domain.py`	`status` defaulted to `"pending"`
`RootCauseCandidate`	`domain.py`	`confidence` bounded
`ActionPlan`, `ApprovalRequest`	`domain.py`	Typed enums throughout
`AuditRecord`	`audit.py`	Lightweight projection for query-friendly persistence
`WorkflowState`	`workflow.py`	Shared graph state; all fields after `request` are `Optional`
`AgentToolRegistryEntry`	`policy.py`	`use_case_allowlist` list for exact-match eligibility
`MCPToolRequest`	`mcp.py`	**Auto-strips nulls, empties, and whitespace** in Pydantic validator

3.2 Implementation actions required

None. These models are the stable contract everything else builds on. Do not add LLM-facing string fields or free-text dumps to these models — they are meant to stay strongly-typed and independently testable.

3.3 Optional enhancements

- Fix `datetime.utcnow()` deprecation warnings by replacing with `datetime.now(datetime.UTC)` throughout. This is cosmetic — no functional impact.
- Migrate `WorkflowState.Config` class-based config to `model_config = ConfigDict(...)` (Pydantic V2 style) to clear the remaining deprecation warning.

```
# In workflow.py, replace:
class Config:
    arbitrary_types_allowed = True

# With:
model_config = ConfigDict(arbitrary_types_allowed=True)
```

---

4. Phase 2 — Teradata DDL and Repository Layer

Files: `app/sql/context/` , `app/sql/queries/` , `app/repositories/`
Status: DDL and SQL templates are complete. Repository *method signatures* are complete. Repository *method bodies* are stubs — all SQL execution is a `# TODO` .

4.1 DDL deployment sequence

Execute in this exact order against your target Teradata system. Use a service account with `CREATE TABLE` and `CREATE VIEW` rights on the `context` database.

```
-- Step 1: Create the context database (if it doesn't exist)
-- Run as DBA:
CREATE DATABASE context FROM DBC
  AS PERMANENT = 10e9,    -- 10 GB; tune to actual data volumes
     SPOOL     = 5e9;

-- Step 2: Grant the application service account access
GRANT SELECT ON context TO ops_copilot_svc;
GRANT INSERT ON context.context_audit TO ops_copilot_svc;
-- Note: INSERT only on context_audit. All other tables are READ-ONLY for the app

-- Step 3: Create tables
.RUN FILE = app/sql/context/create_context_tables.sql

-- Step 4: Create views (depends on tables existing)
.RUN FILE = app/sql/context/create_context_views.sql

-- Step 5: Verify
SELECT TableName, TableKind
FROM DBC.TablesV
WHERE DatabaseName = 'context'
ORDER BY 1;
```

Expected output: 8 tables (T), 5 views (V).

4.2 Seed the tool registry

Before the app can make policy decisions, `context.agent_tool_registry` must have rows. Insert one row per MCP tool you intend to enable:

```
INSERT INTO context.agent_tool_registry
(tool_name, category, description, is_enabled, requires_approval,
 is_read_only, use_case_allowlist_csv)
VALUES
```

```
('dba_workloadHealth', 'dba',
 'Live workload health pull for a TASM/TIWM workload_id.',
 1, 0, 1,
 'workload_analysis,pipeline_incident_diagnosis'),

('sec_userDbPermissions', 'security',
 'Database-level permissions for a named user.',
 1, 0, 1,
 'access_review,security_audit'),

('sec_userRoles', 'security',
 'Roles granted to a named user.',
 1, 0, 1,
 'access_review,security_audit'),

('sec_rolePermissions', 'security',
 'Permissions carried by a named role.',
 1, 0, 1,
 'access_review,security_audit'),

('vector_runbookSearch', 'vector',
 'Semantic search over runbook/SOP corpus.',
 1, 0, 1,
 'pipeline_incident_diagnosis,workload_analysis,access_review,security_audit');
```

> **Note:** `tool_name` values must exactly match the names exposed by Teradata Enterprise MCP. Confirm these before inserting — they are the single source of truth used by both `ToolRegistry` and `policy_service`.

4.3 Wire up repository method bodies

Every repository method body currently contains a `# TODO: execute ...` comment. The wiring pattern is identical for all five methods. Use the following template:

```
# In any repository method, replace the # TODO block with:
def get_data_product(self, data_product_id: str) -> Optional[DataProductSummary]:
    sql = self._load_sql("get_data_product.sql")
    # teradatasql uses qmark paramstyle: replace :param with ?
    sql = sql.replace(":data_product_id", "?")

    cursor = self._conn.cursor()
    try:
        cursor.execute(sql, [data_product_id])
        row = cursor.fetchone()
        if row is None:
            return None
```

```
        cols = [d[0].lower() for d in cursor.description]
        record = dict(zip(cols, row))
        return DataProductSummary(
            data_product_id=record["data_product_id"],
            name=record["name"],
            domain=record.get("domain"),
            description=record.get("description"),
            owner=record["owner"],
            owning_team=record.get("owning_team"),
            business_criticality=record.get("business_criticality"),
        )
    finally:
        cursor.close()
```

Repeat this pattern for each of the six query files. The column names in the `record` dict will match the column aliases in the corresponding `.sql` file.

4.4 Parameter binding style decision

The scaffold SQL files use `:param_name` style for readability. `teradatasql` uses `?` (qmark) paramstyle. Two approaches:

Option A — Replace at load time (shown above): Simple regex replace `:param_name` → `?` and pass values as a positional list. Works for all six query files as written.

Option B — Rewrite SQL files to use `?`: Edit the `.sql` files directly. Better if you plan to use the SQL files as-is in other tooling (e.g. BTEQ scripts).

Pick one and apply it consistently across all repository files.

4.5 Connection pooling recommendation

Do not create a new Teradata connection on every API request. Wire a connection pool in `app/main.py`'s `lifespan` function:

```
# In app/main.py lifespan():
import teradatasql
from app.core.config import get_settings

settings = get_settings()
# teradatasql does not have a built-in pool; use a simple queue-based pool
# or the teradataml SessionContext pattern depending on your platform standards.
conn = teradatasql.connect(
    host=settings.td_host,
    user=settings.td_username,
    password=settings.td_password,
```

```
        database=settings.td_database,
        logmech=settings.td_logmech,
        encryptdata=str(settings.td_encryptdata).lower(),
    )
    app.state.td_conn = conn
    yield
    conn.close()
```

Then update `routes.py`'s `_get_workflow_deps()` to use `request.app.state.td_conn` instead of the `_FakeConn()` placeholder.

---

5. Phase 3 — Workflow Orchestration Layer

Files: `app/workflows/ops_copilot_graph.py`, `app/workflows/nodes/*.py` (10 files)
Status: All node signatures, state I/O contracts, and error handling are complete. Several nodes have `# TODO` stubs for specific integration points (listed below).

5.1 Node completion checklist

Work through each node in pipeline order. Each node's `run()` function is fully wired into the graph — only the noted internal stubs need completing.

Node 1 — `intake.py` (Complete)

No TODOs. The keyword-based classifier and entity-hint extractor work as-is.

Recommended enhancement (after go-live): Replace `extract_entity_hint()` with an LLM-based NER call to handle non-standard phrasing. The function signature and return type stay identical.

Node 2 — `resolve_entity.py` (Complete)

Depends on Phase 2's `get_alias_matches()` being wired up. Threshold constants (`_MIN_AUTO_RESOLVE_CONFIDENCE = 0.85`, `_AMBIGUITY_MARGIN = 0.05`) are in the file — tune them after reviewing real alias data distributions.

Node 3 — `policy_check.py` (Functional with known gap)

TODO: Object-level sensitivity check via `security_repo.get_access_policy()` is commented out pending an entity-to-object-name mapping. Until wired:

- PCI object sensitivity is **not** factored into the execution-mode ceiling.

- The ceiling is determined only by entity-resolution state.

To complete: once `resolved_entity.canonical_id` can be mapped to a Teradata database object name (e.g. via a lookup in `context.data_product_catalog`), add this call inside `_evaluate_internal()`:

```
if entity and entity.canonical_id:
    object_name = self._lookup_object_name(entity.canonical_id)  # TODO: implemen
    if object_name:
        policies = self._security_repo.get_access_policy(object_name)
        for p in policies:
            if p.sensitivity_level in _RESTRICTED_SENSITIVITY_LEVELS:
                effective_ceiling = self._min_status(
                    effective_ceiling, PolicyDecisionStatus.READ_ONLY
                )
```

Node 4 — `retrieve_context.py` (Functional)

Depends on Phase 2 repository wiring. Workload signals are intentionally left empty here (see TODO in file). Complete after a `workload_id ↔ data_product_id` mapping table or join path is defined.

Node 5 — `enrich_with_mcp.py` (Functional with stubs)

Two **TODO** items:

1. `sec_*` tool planning is intentionally omitted — wire once a mechanism exists to extract a target *user identity* from the request that is distinct from the data-product entity. See inline comment for the parameter the tool would need.
2. MCP tool names (`dba_workloadHealth`, `vector_runbookSearch`) are placeholders — confirm exact names against the live MCP catalog and update both this file and `context.agent_tool_registry` seed data.

Node 6 — `score_root_cause.py` (Functional)

Four scoring rules are complete. Two categories (`security_regression`, `schema_drift`) have no rule yet — add them as independent functions following the exact pattern of the four existing rules, then append to `_RULES`.

Node 7 — `generate_response.py` (Functional)

`_llm_summarize()` returns `None` (disabled stub). Wire it in Phase 4/5:

```
def _llm_summarize(pack, candidates):
    from app.services.prompt_service import PromptService
    # Build the prompt
    prompt_svc = PromptService()
    prompt = prompt_svc.get_root_cause_prompt(
        top_category=candidates[0].category.value,
        top_confidence=f"{candidates[0].confidence:.0%}",
        top_evidence_summary=candidates[0].evidence_summary,
        # ... other variables from root_cause_prompt.txt
    )
    # Call Anthropic API
    import anthropic
    client = anthropic.Anthropic()
    message = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=512,
        system=prompt_svc.get_system_prompt(),
        messages=[{"role": "user", "content": prompt}],
    )
    return message.content[0].text
```

Node 8 — `request_approval.py` (Functional)

TODO: Add `ActionType.EXECUTE_FIX_SCRIPT` to `models/domain.py` and `ACTIONS_REQUIRING_APPROVAL` in `approval_service.py` if rerun vs. fix-script need to be tracked distinctly.

Node 9 — `execute_action.py` (Safe actions only)

Three approval-required actions have no executor by design: `RERUN_PIPELINE`, `GRANT_OR_REVOKE_ACCESS`, `EXECUTE_MACRO`.

When you are ready to implement a rerun executor, follow this pattern:

```
def _execute_rerun_pipeline(plan: ActionPlan) -> str:
    """Route through a stored procedure, not free-form SQL."""
    pipeline_id = plan.target_object
    if not pipeline_id:
        raise ValueError("pipeline_id required for rerun")
    # Call a specific stored procedure or Ab Initio restart API —
    # NOT a dynamically generated SQL string from the plan payload.
    # TODO: implement narrow adapter call here
    logger.info("STUB: would rerun pipeline=%s", pipeline_id)
    return f"rerun_initiated:{pipeline_id}"
```

Then add it to `_ACTION_EXECUTORS`:

```
_ACTION_EXECUTORS[ActionType.RERUN_PIPELINE] = _execute_rerun_pipeline
```

Before enabling any approval-required executor in production:

- Add a second policy check immediately before execution (not just trusting the upstream `policy_decision` snapshot which may be stale).
- Confirm the service account has only the minimum permissions required.
- Add a dry-run mode that logs intent without executing.

Node 10 — `audit_result.py` (Complete)

Depends on Phase 2's `audit_repo.insert_audit_record()` being wired up.

5.2 LangGraph migration (optional)

The scaffold uses a plain Python sequential function. To migrate to a real `StateGraph`:

```
from langgraph.graph import StateGraph
from app.models.workflow import WorkflowState

builder = StateGraph(WorkflowState)
builder.add_node("intake", intake.run)
builder.add_node("resolve_entity", lambda s: resolve_entity.run(s, context_repo))
# ... add remaining 8 nodes ...

# Add conditional edge: skip enrichment if clarification required
builder.add_conditional_edges(
    "resolve_entity",
    lambda s: "skip_to_explain" if s.requires_clarification else "policy_check",
)
graph = builder.compile()
```

---

6. Phase 4 — MCP Adapters, Services, and App Layer

Files: `app/mcp/`, `app/services/`, `app/core/`, `app/api/`, `app/main.py`
Status: All interfaces complete. Integration stubs marked with `# TODO`.

6.1 MCP client integration

The `MCPClient` is wired to a `MCPTransport` Protocol. Wire the real Teradata Enterprise MCP SDK transport as follows:

```
# In app/main.py lifespan(), after connection pool setup:
from app.mcp.client import MCPClient
from app.mcp.tool_registry import ToolRegistry
from app.core.config import get_settings

settings = get_settings()

# Step 1: Load tool definitions from context.agent_tool_registry
#         (use the same TD connection from the pool)
cursor = app.state.td_conn.cursor()
cursor.execute("SELECT * FROM context.vw_enabled_tool_registry")
rows = [dict(zip([d[0] for d in cursor.description], r)) for r in cursor.fetchall
cursor.close()

# Step 2: Build the registry and client
registry = ToolRegistry.from_registry_rows(rows)

# Step 3: Wire the real MCP transport
# TODO: replace TeradataMCPSession with actual SDK class name
# from teradata_mcp import TeradataMCPSession  # confirm import path
# transport = TeradataMCPSession(endpoint_url=settings.mcp_endpoint_url)
transport = None  # stub until SDK confirmed

mcp_client = MCPClient(transport=transport, tool_registry=registry,
                       timeout_seconds=settings.mcp_timeout_seconds)
app.state.mcp_client = mcp_client
```

Then update `_get_workflow_deps()` in `routes.py` to pass `app.state.mcp_client`.

6.2 Security adapter — confirm tool names

The three security tool names in `security_tools.py` match the spec exactly:

```
sec_userDbPermissions
sec_userRoles
sec_rolePermissions
```

Before go-live: verify these strings against the live Teradata Enterprise MCP tool catalog. If they differ, update:

1. `app/mcp/adapters/security_tools.py` — the string literals in each method

2. `context.agent_tool_registry` — the `tool_name` column values
3. `app/mcp/tool_registry.py` — `ToolRegistry` entries if pre-seeded in code

6.3 PolicyService — object-level sensitivity

`_get_sensitivity_ceiling()` returns `APPROVED_FOR_SAFE_ACTION` unconditionally (i.e. no PCI ceiling reduction). Complete per the Node 3 instructions in §5.1.

6.4 ApprovalService — durable store

The in-memory `_store` dict loses all pending approvals on process restart. For production, replace with one of:

Option A — Teradata table (preferred, keeps all state on-platform):

```
CREATE MULTISET TABLE context.approval_requests (
    approval_id          VARCHAR(64)   NOT NULL,
    requested_by         VARCHAR(128)  NOT NULL,
    action_type          VARCHAR(64)   NOT NULL,
    target_object        VARCHAR(256),
    requested_payload    CLOB,
    status               VARCHAR(32)   DEFAULT 'pending',
    approved_by          VARCHAR(128),
    approved_ts          TIMESTAMP(6),
    requested_ts         TIMESTAMP(6)  DEFAULT CURRENT_TIMESTAMP(6),
    rejection_reason     VARCHAR(1000)
) PRIMARY INDEX (approval_id);
```

Then rewrite `ApprovalService._store` operations as `audit_repo`-style INSERT/UPDATE calls against this table.

Option B — ServiceNow integration: Wire `create_approval_request()` to open a ServiceNow change request, and `get_approval_status()` to poll the ServiceNow API for current state.

6.5 Authentication — replace stub

`app/core/security.py`'s `_extract_user_id_from_token()` accepts any token ≥ 8 characters. Replace with your Wells Fargo SSO/OIDC integration:

```
def _extract_user_id_from_token(token: str) -> Optional[str]:
    # TODO: call your OIDC introspection endpoint
    import httpx
    resp = httpx.post(
```

```
            "https://your-oidc-provider/introspect",
            data={"token": token, "client_id": "ops-copilot"},
            timeout=5.0,
        )
        if resp.status_code == 200 and resp.json().get("active"):
            return resp.json().get("username")
        return None
```

6.6 API routes — singleton dependencies

`routes.py` currently constructs `ApprovalService()` on every `/approvals` call and a `_FakeConn` on every `/diagnose` call. Both must be singletons stored on `app.state` after the lifespan startup hook runs:

```
# In lifespan():
app.state.approval_service = ApprovalService()
app.state.workflow_deps = WorkflowDependencies(
    context_repo=TeradataContextRepo(app.state.td_conn),
    ops_repo=TeradataOpsRepo(app.state.td_conn),
    security_repo=TeradataSecurityRepo(app.state.td_conn),
    audit_repo=AuditRepo(app.state.td_conn),
    tool_registry_entries=registry_entries,  # from §6.1
    mcp_client=app.state.mcp_client,
)
```

Then update the route handlers to use `request.app.state.*` via a FastAPI `Request` dependency rather than the local factory functions.

---

7. Phase 5 — Prompt Templates and Test Suite

Files: `app/prompts/*.txt` , `app/tests/test_*.py`

7.1 Prompt templates

Four templates are complete and ready for use. Each uses `{variable}` placeholders that `PromptService.render()` fills at runtime.

Template	Used by	Key variables
`system_prompt.txt`	Every LLM API call as `system=`	None (static)

`ops_diagnosis_prompt.txt`	`generate_response.py` via `PromptService`	`entity_id`, `pipeline_health_summary`, `missing_data_flags`, etc.
`security_audit_prompt.txt`	`generate_response.py` for `security_audit` use case	`permissions_summary`, `roles_summary`, `access_policy_summary`
`root_cause_prompt.txt`	`_llm_summarize()` in `generate_response.py`	`top_category`, `top_evidence_summary`, `all_candidates_summary`

To wire a prompt into an LLM call:

```
# In generate_response.py _llm_summarize():
prompt = PromptService().get_root_cause_prompt(
    top_category=candidates[0].category.value,
    top_confidence=f"{candidates[0].confidence:.0%}",
    top_evidence_summary=candidates[0].evidence_summary,
    top_evidence_refs=", ".join(candidates[0].evidence_refs),
    top_missing_evidence=", ".join(candidates[0].missing_evidence) or "none",
    all_candidates_summary="\n".join(
        f"- {c.category.value} ({c.confidence:.0%}): {c.evidence_summary}"
        for c in candidates
    ),
    pipeline_health_summary=_format_pipeline_health(pack),
    workload_signals_summary=_format_workload_signals(pack),
    mcp_enrichment_summary=str(pack.mcp_enrichment),
    runbook_snippets="\n".join(pack.runbook_snippets) or "none available",
)
```

Prompt governance note: treat prompt templates as versioned configuration, not application code. Changes to prompts can change model behaviour meaningfully. Review prompt changes through the same approval process as schema changes.

7.2 Test suite

101 tests across 6 files. All pass against stubbed repositories (no live Teradata or MCP connection required).

```
# Run all tests
python3 -m pytest app/tests/ -v

# Run a single file
```

```
python3 -m pytest app/tests/test_entity_resolution.py -v

# Run with coverage report
python3 -m pytest app/tests/ --cov=app --cov-report=term-missing
```

Extending tests for new use cases

When you add a new use case (e.g. `schema_drift_analysis`), add:

1. A new keyword set to `intake.py`'s `_USE_CASE_KEYWORDS`
2. A new scoring rule in `score_root_cause.py`
3. At minimum three new tests in `test_workflow_diagnosis.py`:
    - Classification test (does intake detect the right use case?)
    - Scoring test (does the rule fire on the right evidence?)
    - End-to-end test (does the workflow produce the right action?)

---

8. Integration Sequence

Complete these steps **in order** for a production integration. Do not skip steps — each step has a verification gate that must pass before proceeding.

Step 1 — Deploy DDL to development Teradata (Phase 2)

```
Gate: SELECT COUNT(*) FROM DBC.TablesV WHERE DatabaseName='context' returns 13 ro
```

Step 2 — Seed context.agent_tool_registry (Phase 2)

```
Gate: SELECT COUNT(*) FROM context.vw_enabled_tool_registry >= 1 row
```

Step 3 — Wire repository method bodies (Phase 2)

```
Gate: python3 -m pytest app/tests/ -v → 101 passed (repositories still use mocks;
      Manual: call TeradataContextRepo.get_data_product('YOUR_REAL_ID') against d
```

Step 4 — Seed context.data_product_catalog and

context.data_product_alias (Phase 2)

Populate with real data products and their known aliases. Minimum viable seed:

```
INSERT INTO context.data_product_catalog VALUES (
  'DP_FIN_DASH', 'Finance Dashboard', 'Finance', 'Daily P&L dashboard',
  'finance-team', 'Finance Analytics', 'critical', '08:00:00',
  'active', 1.0000, CURRENT_TIMESTAMP(6), CURRENT_TIMESTAMP(6), 'dba', 'dba'
);
INSERT INTO context.data_product_alias VALUES (
  'A001', 'DP_FIN_DASH', 'finance dashboard', 'synonym', 1.0000, 'active',
  CURRENT_TIMESTAMP(6), CURRENT_TIMESTAMP(6), 'dba'
);


Gate: EntityResolver.resolve("finance dashboard") returns canonical_id='DP_FIN_DA
```

Step 5 — Wire MCP client transport (Phase 4)

```
Gate: MCPClient(transport=real_transport, ...).call_tool('sec_userRoles', {'user_
      returns MCPToolResult(status=SUCCESS) with a non-empty raw_result
```

Step 6 — Run full workflow against dev environment

```
from app.workflows.ops_copilot_graph import run_workflow, WorkflowDependencies
# ... build real deps with live connection and mcp_client ...
state = run_workflow(UserRequest(
    request_id="integration-test-1",
    requesting_user="your_id",
    raw_text='Why did the "finance dashboard" miss the 8 AM SLA?',
), deps)
print(state.current_stage)        # should be 'complete'
print(state.user_explanation)     # should be a grounded explanation
print(state.root_cause_candidates)


Gate: state.current_stage == 'complete', user_explanation is non-empty, errors ==
```

Step 7 — Start the FastAPI server

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

```
Gate: GET http://localhost:8000/api/v1/health → {"status": "ok"}
      POST http://localhost:8000/api/v1/diagnose with valid Bearer token → 200 wi
```

Step 8 — Wire authentication

```
Gate: POST /api/v1/diagnose with an invalid token → 401
      POST /api/v1/diagnose with a valid SSO token → 200 with requesting_user set
```

Step 9 — Validate audit records

```
SELECT TOP 5 * FROM context.context_audit ORDER BY created_ts DESC;
```

```
Gate: rows appear for each POST /diagnose call; tool_sequence_json and top_root_c
```

---

9. TODO Completion Checklist

Consolidated list of all `# TODO` items across the scaffold, in priority order for a production integration:

Priority 1 — Required for any live call to succeed

- **All 5 repository method bodies** — wire `cursor.execute()` and row-to-model mapping in `teradata_context_repo.py`, `teradata_ops_repo.py`, `teradata_security_repo.py`, `audit_repo.py`
- **Connection pool** — replace `_FakeConn` in `routes.py` with real `teradatasql` connection managed in `app/main.py` lifespan
- **MCP transport** — replace `transport=None` in `MCPClient` constructor with real `TeradataMCPSession` (or equivalent) from the TD Enterprise MCP SDK
- **MCP tool names** — confirm `dba_workloadHealth`, `vector_runbookSearch`, `sec_userDbPermissions`, `sec_userRoles`, `sec_rolePermissions` against live MCP catalog; update files + DB seed data if they differ

Priority 2 — Required for correct policy and security behaviour

- **Authentication** — replace `_extract_user_id_from_token()` stub in `core/security.py` with real OIDC/SSO validation

- **Object-level sensitivity check** in `policy_service._get_sensitivity_ceiling()` — map `canonical_id` to object name, call `security_repo.get_access_policy()`
- **ApprovalService durable store** — replace in-memory `_store` with Teradata table or ServiceNow integration
- **Singleton dependencies** in `routes.py` — move `WorkflowDependencies` and `ApprovalService` construction to `app.state` in lifespan hook

Priority 3 — Required for full diagnostic capability

- **LLM summarization** — wire `_llm_summarize()` in `generate_response.py` using `PromptService` and Anthropic SDK
- **Security tool planning** in `enrich_with_mcp.py` — wire `sec_*` calls once target-user-identity extraction from the request is implemented
- **Workload signal retrieval** in `retrieve_context.py` — implement `workload_id ↔ data_product_id` mapping and populate `context_pack.workload_signals`
- **Fallback catalog search** in `entity_resolver._fallback_catalog_search()` — add direct name-fragment search when alias lookup returns no candidates

Priority 4 — Recommended before full production rollout

- **Rerun pipeline executor** in `execute_action.py` — narrow stored-procedure adapter, not free-form SQL; requires separate security review
- **`security_regression` scoring rule** in `score_root_cause.py` — depends on MCP security tool results being populated in `context_pack.mcp_enrichment`
- **`schema_drift` scoring rule** — depends on schema-comparison evidence source
- **`ActionType.EXECUTE_FIX_SCRIPT`** — add to `models/domain.py` enum and `ACTIONS_REQUIRING_APPROVAL` frozenset if needed
- **Dependency depth configurability** — source `_DEFAULT_DEPENDENCY_DEPTH` from `settings.dependency_chain_max_depth` rather than hardcoding
- **Entity resolution threshold tuning** — review `_MIN_AUTO_RESOLVE_CONFIDENCE` (0.85) and `_AMBIGUITY_MARGIN` (0.05) against real alias data distributions
- **`datetime.utcnow()` deprecation** — replace with `datetime.now(datetime.UTC)`

Priority 5 — Future enhancements

- **Replay / evaluation store** — persist full `WorkflowState` snapshot alongside `AuditRecord` in `audit_result.py`; key by `request_id`
- **LangGraph migration** — wrap `run_workflow` nodes as a `StateGraph` for conditional edge support, checkpointing, and distributed replay
- **NER-based entity extraction** — replace keyword-based `extract_entity_hint()` in `intake.py` with a structured NER pass
- **`context.approval_requests` table** — normalise ApprovalService storage into a dedicated Teradata table (SQL stub in §6.4)
- **Python-json-logger** — install for JSON log output; current fallback is plain text which is fine for development

---

10. Configuration Reference

All settings live in `app/core/config.py` and are bound from environment variables. Defaults shown permit stub-mode operation with no live connections.

Variable	Default	Required for live?	Notes
`TD_HOST`	`localhost`	Yes	Teradata server hostname
`TD_USERNAME`	`""`	Yes	Service account username
`TD_PASSWORD`	`""`	Yes	Source from secrets manager
`TD_DATABASE`	`context`	Yes	Target schema for all context.* objects
`TD_LOGMECH`	`TD2`	No	`TD2` / `LDAP` / `KRB5` / `JWT`
`TD_ENCRYPTDATA`	`true`	No	PCI best practice: always `true`
`TD_CONNECT_TIMEOUT`	`30`	No	Seconds

MCP_ENDPOINT_URL	""	Yes (MCP)	Leave blank for stub mode
MCP_TIMEOUT_SECONDS	30.0	No	Per-call timeout
ENTITY_MIN_AUTO_RESOLVE_CONFIDENCE	0.85	No	Tune after alias data review
ENTITY_AMBIGUITY_MARGIN	0.05	No	Tune after alias data review
MIN_ACTION_CONFIDENCE	0.60	No	Below this, proposes diagnostics not a fix
DEPENDENCY_CHAIN_MAX_DEPTH	3	No	Increase for deep dependency analysis
ANTHROPIC_API_KEY	""	Yes (LLM)	Leave blank to disable LLM summarization
LLM_MODEL	claude-sonnet-4-6	No	Any Anthropic model string
LLM_MAX_TOKENS	1024	No	
APP_PORT	8000	No	
DEBUG	false	No	true enables plain-text logging
LOG_LEVEL	INFO	No	DEBUG for development

---

11. Extending the Framework

Adding a new use case

1. **Add the use case name** to UseCaseName enum in models/domain.py

2. **Add keyword triggers** to `_USE_CASE_KEYWORDS` in `workflows/nodes/intake.py`
3. **Seed tool eligibility** — insert rows into `context.agent_tool_registry` with the new use case name in `use_case_allowlist_csv`
4. **Add a scoring rule** in `workflows/nodes/score_root_cause.py` if the new use case has a distinct root-cause pattern
5. **Add a prompt template** in `app/prompts/` and a `PromptService` method in `services/prompt_service.py` if it needs distinct LLM formatting
6. **Add tests** — minimum: classification test, scoring test, end-to-end test

Adding a new MCP tool

1. **Insert a row** into `context.agent_tool_registry` with exact tool name, category, `use_case_allowlist_csv`, and `requires_approval` flag
2. **Add a `MCPToolDefinition`** to the registry seed (or let `ToolRegistry.from_registry_rows()` load it automatically from the DB)
3. **Add a method** to the appropriate category adapter (`DBAToolsAdapter`, `SecurityToolsAdapter`, etc.) using the `build_request()` + `_safe_invoke()` pattern
4. **Wire the call** in `enrich_with_mcp.py`'s `_plan_tool_calls()` when the conditions for calling it are met

Adding a new entity type

1. **Add the entity type** to `EntityType` enum in `models/domain.py`
2. **Create alias rows** in `context.data_product_alias` (or a parallel alias table if pipelines/users need separate alias tables)
3. **Update `get_alias_matches.sql`** to join the new alias source if needed
4. **Update `resolve_entity.py`** if entity-type-specific resolution logic is needed beyond the shared confidence thresholding

Adding a new root-cause scoring rule

Follow the exact pattern of any existing rule in `score_root_cause.py`:

```
def _score_schema_drift(pack: ContextPack) -> RootCauseCandidate | None:
    """Return a candidate if evidence of schema drift exists in the pack."""
    # Check for evidence in pack.mcp_enrichment, pack.pipeline_health, etc.
```

```
    schema_evidence = pack.mcp_enrichment.get("schema_comparison_tool")
    if not schema_evidence:
        return None
    return RootCauseCandidate(
        category=RootCauseCategory.SCHEMA_DRIFT,
        confidence=0.75,
        evidence_summary=f"Schema comparison detected drift: {schema_evidence[:20
        evidence_refs=["mcp_enrichment.schema_comparison_tool"],
        missing_evidence=[],
    )

# Then append to _RULES:
_RULES = [
    _score_failed_etl_job,
    _score_upstream_data_delay,
    _score_workload_saturation,
    _score_dashboard_refresh_failure,
    _score_schema_drift,  # <-- new
]
```

---

12. Troubleshooting

Tests pass but real Teradata call returns empty results

- Check service account has `SELECT` on `context.*` views (not just base tables)
- Verify `TD_DATABASE=context` is set; if default database differs, qualify all object references as `context.table_name` explicitly
- Run `SELECT * FROM context.vw_latest_pipeline_status` directly in BTEQ/SQL Assistant to confirm rows exist

`MCPClientError: tool 'X' is unknown or disabled`

- Tool name in the code does not exactly match `context.agent_tool_registry.tool_name`
- Check for trailing spaces or case differences: `SELECT tool_name, LENGTH(tool_name) FROM context.agent_tool_registry`
- Verify `is_enabled = 1` for the tool row

`PolicyDecision.status = BLOCKED` unexpectedly

- Policy evaluation threw an exception (fails closed by design)
- Check `state.errors` list — the exception message is appended there
- Enable `DEBUG` logging to see the full traceback from `policy_service.py`

`requires_clarification = True` for a known entity

- The alias text in `context.data_product_alias` does not match what the user typed
- Check `get_alias_matches.sql` — matching is case-insensitive LIKE; punctuation differences (dash vs space) may cause misses
- Lower `ENTITY_MIN_AUTO_RESOLVE_CONFIDENCE` temporarily to diagnose, then add better alias rows rather than lowering the threshold permanently

`action_outcome = skipped_not_approved` when you expected execution

- `approval_request.status` is `PENDING` — approval has not been granted
- In the in-memory `ApprovalService`, call `record_decision(approval_id, user, APPROVED)` explicitly in your test/integration harness
- In production, ensure the approval callback/webhook updates the durable store before the execute node runs

`audit_write_failed` in `fallback_paths_taken`

- `audit_repo.insert_audit_record()` threw — most likely a Teradata connection error
- The workflow result itself is still valid; only the audit row is missing
- Check TD connection health; the `context_audit` table must exist and the service account must have `INSERT` permission on it

---

13. Scaffold Status Summary

Layer	Files	Status	Key remaining work
Models	5	✅ Production-ready	Minor Pydantic V2 style updates
Teradata DDL	2	✅ Production-ready	Deploy to target environment
SQL query			

templates	6	✅ Production-ready	Confirm bind-param style
Repositories	4	🔶 Signatures complete, bodies stubbed	Wire `cursor.execute()` per §4.3
Workflow graph	1	✅ Production-ready	Optional LangGraph migration
Workflow nodes	10	🔶 Mostly complete	See node checklist §5.1
MCP registry	1	✅ Production-ready	—
MCP client	1	🔶 Complete except transport	Wire real MCP SDK per §6.1
MCP adapters	4	✅ Production-ready	Confirm exact tool names
Policy service	1	🔶 Tool eligibility complete	Add sensitivity ceiling per §6.3
Approval service	1	🔶 Complete, in-memory store	Add durable store per §6.4
Entity resolver	1	✅ Production-ready	Tune thresholds post-alias-data-review
Context engine	1	✅ Production-ready	—
Root cause service	1	✅ Production-ready	—
Prompt service	1	✅ Production-ready	—
Core config/logging	2	✅ Production-ready	—
Core security	1	🔶 Stub auth only	Wire SSO/OIDC per §6.5
API routes	1	🔶 Functional, fake deps	Wire singletons per §6.6
App main	1	🔶 Lifespan hook stubbed	Add connection pool + state per §4.5
Prompt templates	4	✅ Production-ready	Review/tune prompt text
			Extend per §7.2 as use cases

Test suite	6	✅ 101/101 passing	expand

Legend: ✅ = production-ready as-is · 🔶 = complete interface, integration work remaining